MODULE *sync_scheduler*

Version 2 block-download scheduler specification.

Models the layer above the per-connection protocol in *protocol.tla*: the node choosing which peer to ask for each block — the layer where Zebra's genesis-to-tip sync stall lived in the legacy protocol (*ZcashFoundation*/zebra ≠ 10679; see .. /*sync_scheduler.tla*). Zebra's *v*2 stack defers this layer to a future scheduler ("ibd-engine"), so this module models it ahead of the implementation.

The *v*2 protocol gives a request more ways to end without a block than the legacy protocol did. Only one of them says anything about the peer's chain:

not-found	the per-entry result 0*x*02 of get-blocks: an explicit statement that the peer lacks the block.
REFUSED	the responder reset the stream, declining to serve it (overload, rate limits, Zebra's bulk-stream cap). Says nothing about the peer's chain.
truncation	the responder finished after any complete entry, leaving later entries unanswered ("get-blocks", "get-block-range"). Routine and sanctioned; says nothing about the peer's chain.
timeout	no response within the stall timeout; the requester cancels with *CANCELLED* ("Block Download Parameters"). Says nothing about the peer's chain.

Three booleans mark each of the uninformative outcomes as informative, each reproducing a variant of the legacy registry-poisoning bug:

TreatRefusedAsMissing = TRUE →	a REFUSED marks the peer missing
TreatTruncatedAsMissing = TRUE →	an unanswered entry marks the peer missing
BuggyTimeout = TRUE →	a timeout marks the peer missing (the exact legacy zebra ≠ 10679 bug)

Separately, the scheduler applies Zebra's unresponsive-peer rule: a peer that times out *UnresponsiveLimit* requests in a row, with no response of any kind in between, is disconnected (*ZcashFoundation*/zebra ≠ 11276). The *Redial* boolean decides whether disconnected peers are ever redialled; without it, evicting the only holder of a block stalls the sync even though every registry entry is honest.

RegistryHonest and *EventuallyAllVerified* hold with every switch in its fixed position (*Redial* = TRUE, the rest FALSE) and are violated under the buggy settings, each as a short *TLC* counterexample.

See .. /documents/v2-*modeling.md* for the full write-up.

EXTENDS *Naturals*, *FiniteSets*

CONSTANTS

set of peer ids
@type: *Set*(*Str*);
Peers,

blocks to download are 1 .. *MaxBlock*
@type: *Int*;
MaxBlock,

peers that hold only blocks 1 .. *LagTip*
@type: *Set*(*Str*);
LaggingPeers,

chain height of a lagging peer (< *MaxBlock*)
@type: *Int*;

LagTip,
bound on re-queue attempts per block after a notfound
@type: *Int*;
MaxRetries,
consecutive timeouts before a peer is disconnected
@type: *Int*;
UnresponsiveLimit,
TRUE treats a REFUSED as a notfound (buggy)
@type: *Bool*;
TreatRefusedAsMissing,
TRUE treats an unanswered entry as a notfound (buggy)
@type: *Bool*;
TreatTruncatedAsMissing,
TRUE treats a timeout as a notfound (legacy bug)
@type: *Bool*;
BuggyTimeout,
TRUE redials disconnected peers
@type: *Bool*;
Redial

$Blocks \triangleq 1 \,..\, MaxBlock$
$NONE \triangleq$ "none" sentinel for "no peer"; must not be a member of *Peers*

Ground truth: a lagging peer holds blocks up to *LagTip*; every other peer is at tip and holds them all. The scheduler never reads this directly — it learns a peer's contents only through the outcomes below.
$PeerTip(p) \triangleq$ IF $p \in LaggingPeers$ THEN $LagTip$ ELSE $MaxBlock$
$Holds(p,\ b) \triangleq b \leq PeerTip(p)$

VARIABLES
@type: *Set*(*Int*);
verified, subset of *Blocks* downloaded and verified (the frontier)
@type: *Set*(*Int*);
pending, subset of *Blocks* the scheduler still wants and may request
@type: *Int* → *Str*;
inflight, [*Blocks* → *Peers* ∪ {*NONE*}] peer a block is requested from
@type: *Str* → (*Int* → *Str*);
avail, [*Peers* → [*Blocks* → {"has", "missing", "unknown"}]] registry
@type: *Int* → *Int*;
retries, [*Blocks* → 0 .. *MaxRetries*] re-queue attempts used
@type: *Str* → *Int*;
timeouts, [*Peers* → 0 .. *UnresponsiveLimit*] consecutive unanswered timeouts
@type: *Str* → *Bool*;
connected [*Peers* → BOOLEAN] FALSE once disconnected as unresponsive

$vars \triangleq \langle verified,\ pending,\ inflight,\ avail,\ retries,\ timeouts,\ connected \rangle$

$Init \triangleq$
$\quad \wedge\ verified = \{\}$
$\quad \wedge\ pending = Blocks$
$\quad \wedge\ inflight = [b \in Blocks \mapsto NONE]$
$\quad \wedge\ avail = [p \in Peers \mapsto [b \in Blocks \mapsto \text{“unknown”}]]$
$\quad \wedge\ retries = [b \in Blocks \mapsto 0]$
$\quad \wedge\ timeouts = [p \in Peers \mapsto 0]$
$\quad \wedge\ connected = [p \in Peers \mapsto \text{TRUE}]$

The scheduler opens a get-blocks request for a pending block toward a connected peer not already marked missing for it.

$Request \triangleq$
$\quad \exists\, b \in pending :$
$\quad\quad \exists\, p \in Peers :$
$\quad\quad\quad \wedge\ inflight[b] = NONE$
$\quad\quad\quad \wedge\ connected[p]$
$\quad\quad\quad \wedge\ avail[p][b] \neq \text{“missing”}$
$\quad\quad\quad \wedge\ inflight' = [inflight \text{ EXCEPT } ![b] = p]$
$\quad\quad\quad \wedge\ pending' = pending \setminus \{b\}$
$\quad\quad\quad \wedge\ \text{UNCHANGED } \langle verified, avail, retries, timeouts, connected\rangle$

The in-flight peer genuinely has the block: it is delivered and verified. Any response resets the peer's consecutive-timeout count.

$DeliverBlock \triangleq$
$\quad \exists\, b \in Blocks :$
$\quad\quad \text{LET } p \triangleq inflight[b] \text{ IN}$
$\quad\quad\ \wedge\ p \neq NONE$
$\quad\quad\ \wedge\ Holds(p,\ b)$
$\quad\quad\ \wedge\ verified' = verified \cup \{b\}$
$\quad\quad\ \wedge\ avail' = [avail \text{ EXCEPT } ![p][b] = \text{“has”}]$
$\quad\quad\ \wedge\ inflight' = [inflight \text{ EXCEPT } ![b] = NONE]$
$\quad\quad\ \wedge\ timeouts' = [timeouts \text{ EXCEPT } ![p] = 0]$
$\quad\quad\ \wedge\ \text{UNCHANGED } \langle pending, retries, connected\rangle$

The in-flight peer genuinely lacks the block and answers the entry with the explicit not-found result. This is the one outcome that legitimately marks the registry, and the block is re-queued (bounded) toward other peers.

$NotFound \triangleq$
$\quad \exists\, b \in Blocks :$
$\quad\quad \text{LET } p \triangleq inflight[b] \text{ IN}$
$\quad\quad\ \wedge\ p \neq NONE$
$\quad\quad\ \wedge\ \neg Holds(p,\ b)$
$\quad\quad\ \wedge\ inflight' = [inflight \text{ EXCEPT } ![b] = NONE]$
$\quad\quad\ \wedge\ avail' = [avail \text{ EXCEPT } ![p][b] = \text{“missing”}]$

```
    ∧ timeouts' = [timeouts EXCEPT ![p] = 0]
    ∧ IF retries[b] < MaxRetries
        THEN ∧ pending' = pending ∪ {b}
             ∧ retries' = [retries EXCEPT ![b] = @ + 1]
        ELSE ∧ UNCHANGED pending      retries exhausted
             ∧ UNCHANGED retries
    ∧ UNCHANGED ⟨verified, connected⟩
```

The responder resets the request stream with REFUSED — it declines to serve the request, whether or not it holds the block (draft: "Request Streams"; Zebra refuses beyond 2 concurrent bulk streams). A REFUSED is a response, so the timeout count resets; what it does to the registry is the *TreatRefusedAsMissing* switch.

```
Refused ≜
  ∃ b ∈ Blocks :
    LET p ≜ inflight[b] IN
    ∧ p ≠ NONE
    ∧ inflight' = [inflight EXCEPT ![b] = NONE]
    ∧ pending' = pending ∪ {b}
    ∧ timeouts' = [timeouts EXCEPT ![p] = 0]
    ∧ avail' = [avail EXCEPT ![p][b] =
                  IF TreatRefusedAsMissing THEN "missing" ELSE @]
    ∧ UNCHANGED ⟨verified, retries, connected⟩
```

The responder finishes the stream early, leaving this entry unanswered (draft: "get-blocks" / "get-block-range" allow finishing after any complete entry; truncation is resumable). Sanctioned and routine — but uninformative about the peer's chain. The registry effect is the *TreatTruncatedAsMissing* switch.

```
Truncated ≜
  ∃ b ∈ Blocks :
    LET p ≜ inflight[b] IN
    ∧ p ≠ NONE
    ∧ inflight' = [inflight EXCEPT ![b] = NONE]
    ∧ pending' = pending ∪ {b}
    ∧ timeouts' = [timeouts EXCEPT ![p] = 0]
    ∧ avail' = [avail EXCEPT ![p][b] =
                  IF TreatTruncatedAsMissing THEN "missing" ELSE @]
    ∧ UNCHANGED ⟨verified, retries, connected⟩
```

No response within the stall timeout: the requester cancels the stream with *CANCELLED* and re-queues the block (draft: "Block Download Parameters"). The registry effect is the *BuggyTimeout* switch — the exact legacy bug. The peer's consecutive-timeout count rises; at *UnresponsiveLimit* the peer is disconnected as unresponsive (Zebra's rule).

```
Timeout ≜
```

$\exists\, b \in \mathit{Blocks} :$
$\quad$ LET $p \triangleq \mathit{inflight}[b]$ IN
$\quad\quad \wedge\ p \neq \mathit{NONE}$
$\quad\quad \wedge\ \mathit{inflight}' = [\mathit{inflight}$ EXCEPT $![b] = \mathit{NONE}]$
$\quad\quad \wedge\ \mathit{pending}' = \mathit{pending} \cup \{b\}$
$\quad\quad \wedge\ \mathit{avail}' = [\mathit{avail}$ EXCEPT $![p][b] =$
$\quad\quad\quad\quad$ IF $\mathit{BuggyTimeout}$ THEN "missing" ELSE $@]$
$\quad\quad \wedge\ \mathit{timeouts}' = [\mathit{timeouts}$ EXCEPT $![p] =$ IF $@ < \mathit{UnresponsiveLimit}$ THEN $@ + 1$ ELSE $@]$
$\quad\quad \wedge\ \mathit{connected}' = [\mathit{connected}$ EXCEPT $![p] = @ \wedge \mathit{timeouts}[p] + 1 < \mathit{UnresponsiveLimit}]$
$\quad\quad \wedge$ UNCHANGED $\langle \mathit{verified}, \mathit{retries} \rangle$

A disconnected peer is redialled (Zebra maintains connections to its initial and discovered peers, redialling them when they fail). Gated by the *Redial* switch; without it eviction is forever.

$\mathit{Reconnect} \triangleq$
$\quad \exists\, p \in \mathit{Peers} :$
$\quad\quad \wedge\ \mathit{Redial}$
$\quad\quad \wedge\ \neg \mathit{connected}[p]$
$\quad\quad \wedge\ \mathit{connected}' = [\mathit{connected}$ EXCEPT $![p] =$ TRUE$]$
$\quad\quad \wedge\ \mathit{timeouts}' = [\mathit{timeouts}$ EXCEPT $![p] = 0]$
$\quad\quad \wedge$ UNCHANGED $\langle \mathit{verified}, \mathit{pending}, \mathit{inflight}, \mathit{avail}, \mathit{retries} \rangle$

$\mathit{Next} \triangleq$
$\quad \vee\ \mathit{Request}$
$\quad \vee\ \mathit{DeliverBlock}$
$\quad \vee\ \mathit{NotFound}$
$\quad \vee\ \mathit{Refused}$
$\quad \vee\ \mathit{Truncated}$
$\quad \vee\ \mathit{Timeout}$
$\quad \vee\ \mathit{Reconnect}$

Strong fairness on the progress actions encodes the real-world assumption that the node keeps trying, a willing peer eventually responds, and the redial loop keeps running. Refused, *Truncated* and *Timeout* get no fairness — they are adversarial but never forced.

$\mathit{Spec} \triangleq$
$\quad \mathit{Init}$
$\quad \wedge\ \Box[\mathit{Next}]_{\mathit{vars}}$
$\quad \wedge\ \mathrm{SF}_{\mathit{vars}}(\mathit{Request})$
$\quad \wedge\ \mathrm{SF}_{\mathit{vars}}(\mathit{DeliverBlock})$
$\quad \wedge\ \mathrm{SF}_{\mathit{vars}}(\mathit{NotFound})$
$\quad \wedge\ \mathrm{SF}_{\mathit{vars}}(\mathit{Reconnect})$

Liveness: every block is eventually downloaded and verified.

$$EventuallyAllVerified \triangleq \Diamond(verified = Blocks)$$

Safety invariants.

$$
\begin{array}{l}
TypeOK \triangleq \\
\quad \land \quad verified \subseteq Blocks \\
\quad \land \quad pending \subseteq Blocks \\
\quad \land \quad inflight \in [Blocks \to Peers \cup \{NONE\}] \\
\quad \land \quad avail \in [Peers \to [Blocks \to \{\text{“has”},\ \text{“missing”},\ \text{“unknown”}\}]] \\
\quad \land \quad retries \in [Blocks \to 0\ ..\ MaxRetries] \\
\quad \land \quad timeouts \in [Peers \to 0\ ..\ UnresponsiveLimit] \\
\quad \land \quad connected \in [Peers \to \text{BOOLEAN }]
\end{array}
$$

The registry never marks a peer missing for a block it actually holds: only the explicit not-found result may mark it. “A refusal, a truncated response, and a timeout are not notfounds.”

$$
\begin{array}{l}
RegistryHonest \triangleq \\
\quad \forall\, p \in Peers : \\
\quad\quad \forall\, b \in Blocks : \\
\quad\quad\quad avail[p][b] = \text{“missing”} \Rightarrow \neg Holds(p,\ b)
\end{array}
$$

A peer is only ever recorded as having a block it genuinely holds.

$$
\begin{array}{l}
RegistryHasIsSound \triangleq \\
\quad \forall\, p \in Peers : \\
\quad\quad \forall\, b \in Blocks : \\
\quad\quad\quad avail[p][b] = \text{“has”} \Rightarrow Holds(p,\ b)
\end{array}
$$

A verified block was genuinely available from some peer.

$$
\begin{array}{l}
VerifiedAreReal \triangleq \\
\quad \forall\, b \in verified : \\
\quad\quad \exists\, p \in Peers : Holds(p,\ b)
\end{array}
$$
